Universidade Estadual de Campinas

School of Electrical and Computer Engineering (FEEC)

Seminars in Computer Engineering

IA382A - 2025S2

Automated Quality Assessment of AI-Generated Educational Content: A Case Study on Technical Reports

Final Class Project

Leonardo Rangel Alves

November 12, 2025

Contents

Abstract

This project successfully demonstrated a proof-of-concept for an automated pipeline to generate and evaluate AI-produced educational content.

NotebookLM proved to be a powerful tool for content generation, capable of producing a well-structured and highly coherent technical report from a single transcript and a concise prompt. The generated text exhibited a sophisticated understanding of the source material, though it was not without factual errors.

The **Gemini API**, leveraged through a custom Python script, served as a capable and nuanced automated evaluator. It was able to perform sophisticated, metric-based analysis, providing not only quantitative scores but also detailed, qualitative justifications for its assessments. The evaluation highlighted specific, actionable areas for improvement in the generated report, showcasing the potential of LLMs for scalable quality assurance.

The primary takeaway is that while generative AI can significantly accelerate content creation, it requires a robust verification and evaluation framework. An automated, multi-metric approach, as demonstrated in this project, provides a scalable and effective solution for ensuring the quality and accuracy of AI-assisted educational workflows.

1 Introduction

The proliferation of large language models (LLMs) presents a transformative opportunity for academic and educational content creation. This project, inspired by the seminar ”Talking to Machines: A Practical Introduction to Generative AI,” [1] explores a complete pipeline for generating and automatically evaluating educational content using generative AI tools.

While tools like Google’s NotebookLM can rapidly generate structured documents from source materials, a critical challenge remains: ensuring the quality, accuracy, and educational value of this synthesized content. Manual evaluation is subjective and time-consuming, creating a bottleneck for leveraging these technologies at scale.

This project addresses that challenge by proposing and implementing a framework for the automated quality assessment of AI-generated materials. The primary objective is to develop and test a methodology for programmatically evaluating a technical report generated by NotebookLM, using a custom Python script that leverages the Google Gemini API for its analytical capabilities.

2 Methodology

The methodology was executed in two distinct phases: content generation using NotebookLM [2] and automated evaluation using a custom Python script developed with the assistance of the Gemini CLI [3].

2.1 Phase 1: Content Generation with NotebookLM

The first phase involved generating a comprehensive technical report based on the transcript of the seminar ”Talking to Machines: A Practical Introduction to Generative AI.” Google’s NotebookLM [2] was selected as the primary generation tool. The full seminar transcript was loaded as a source into NotebookLM.

To guide the content generation, the following custom-engineered prompt was used:

```
Based on the provided seminar, draft a comprehensive technical
    report that synthesizes the fundamental concepts,
   architectures, and philosophical considerations of
   generative AI. The objective is to create an informative
   document for professionals seeking to deepen their
   understanding of the underlying mechanisms and the broader
   implications of these technologies.
```

This prompt instructed the model to create a high-level synthesis of the material, focusing on both the technical and philosophical aspects, resulting in the document `technical_report_notebooklm.md`.

2.2 Phase 2: Automated Evaluation with Python and Gemini

The second phase focused on creating an automated system to evaluate the quality of the generated report. This was achieved by developing a Python script (`evaluate_report.py`) designed to interact with the Gemini API.

2.2.1 Planning and Implementation

The development process began with a detailed plan (`python_script_plan.md, see in Project Files link`) that outlined the script's objective, architecture, and core components. The plan specified the use of the `google-generativeai` library, environment variables for API key management, and a modular structure for handling file I/O and API interaction.

The implementation of this plan was carried out with the assistance of the **Gemini CLI**. The CLI agent was used to:

- Scaffold the project structure.
- Write the Python code for `evaluate_report.py` based on the plan.
- Create and manage dependencies in `requirements.txt`.
- Debug the script by identifying and fixing errors related to API model names and rate limits.

The final script accepts the paths to the generated report and the source transcript as inputs. It then calls the Gemini API with a series of specialized prompts, each designed to assess the report against a predefined quality metric.

2.2.2 Evaluation Metrics and Prompts

The script evaluates the report based on four key metrics. For each metric, a specific prompt instructs the Gemini model to act as a specialized evaluator and return a score (1-10) and a detailed justification in JSON format.

The prompts for each metric are as follows:

1. **Clarity and Coherence:**

```
As an Academic Reviewer, evaluate the clarity and
   coherence of the following report.
Is the language clear and unambiguous? Does the document
   flow logically from one section to the next?
Return a JSON object with 'score' (integer 1-10) and '
   justification' (string).
```

2. **Factual Accuracy:**

```
As a Fact-Checker, you will be given a source transcript
   and a technical report.
Your task is to verify the factual accuracy of the report
   against the transcript.
Identify any discrepancies. Return a JSON object with '
   score' (integer 1-10) and 'justification' (string)
   listing any errors found.
```

3. **Depth of Analysis:**

```
As a University Professor, assess the depth of analysis in
    this report.
Does it offer critical insights, implications, or
   connections to broader themes as a graduate-level
   document should?
Return a JSON object with 'score' (integer 1-10) and '
   justification' (string).
```

4. **Structural Adherence:**

```
As an Academic Journal Editor, evaluate the report's
   adherence to a formal academic structure.
Is it well-formatted and does it contain all the necessary
    components of a technical report (e.g., Title,
   Abstract, logical sections)?
Return a JSON object with 'score' (integer 1-10) and '
   justification' (string).
```

2.3 Project Files

All project files, including the source code, generated reports, prompts and documentation, can be downloaded from the following link: `https://drive.google.com/`

drive/folders/1JHp41_uldpTGGTAxSeRzE7983Phq9YqZ?usp=sharing

3 Results

The Python script was executed to evaluate the `technical_report_notebooklm.md` against the source transcript. The script successfully generated a detailed evaluation report (`evaluation_results.md`), assigning an **Overall Average Score of 7.25/10**.

The breakdown of the results for each metric is as follows:

3.1 Clarity and Coherence: 10/10

The Gemini evaluator awarded a perfect score, noting that the report was "exceptionally clear and coherent." It praised the use of precise language, effective analogies, and a flawless logical structure that successfully weaves technical and philosophical concepts into a unified narrative.

3.2 Factual Accuracy: 5/10

This was the lowest-scoring category. The evaluator found that while the report was a "well-structured and mostly accurate summary," it contained "several notable factual errors that undermine its credibility." The most significant error was misidentifying the speaker's name and gender. Other inaccuracies included altering specific examples and misquoting technical terms.

3.3 Depth of Analysis: 8/10

The report received a high score for its analytical depth. The evaluator highlighted its ability to move beyond mere description to offer critical insights, particularly through the introduction of interpretive frameworks like the "Shannon Divide" and the "Turing Trap." The primary criticism was its reliance on a single source, which prevented it from achieving a top-tier score that would require synthesizing multiple perspectives.

3.4 Structural Adherence: 6/10

The evaluator noted the report's strong, logical internal structure but penalized it for two "critical structural deficiencies." The most significant omission was the complete lack of a **'References' or 'Bibliography' section**. The second was the absence of a dedicated **'Abstract'**. These omissions rendered the document incomplete as a formal academic paper.

3.5 Project Files

All project files, including the source code, generated reports, prompts and documentation, can be downloaded from the following link: `https://drive.google.com/drive/folders/1JHp41_uldpTGGTAxSeRzE7983Phq9YqZ?usp=sharing`

4 Conclusions

This project successfully demonstrated a proof-of-concept for an automated pipeline to generate and evaluate AI-produced educational content.

NotebookLM proved to be a powerful tool for content generation, capable of producing a well-structured and highly coherent technical report from a single transcript and a concise prompt. The generated text exhibited a sophisticated understanding of the source material, though it was not without factual errors.

The **Gemini API**, leveraged through a custom Python script, served as a capable and nuanced automated evaluator. It was able to perform sophisticated, metric-based analysis, providing not only quantitative scores but also detailed, qualitative justifications for its assessments. The evaluation highlighted specific, actionable areas for improvement in the generated report, showcasing the potential of LLMs for scalable quality assurance.

The primary takeaway is that while generative AI can significantly accelerate content creation, it requires a robust verification and evaluation framework. An automated, multi-metric approach, as demonstrated in this project, provides a scalable and effective solution for ensuring the quality and accuracy of AI-assisted educational workflows.

5 References

[1] G. Sude, "Talking to machines: A practical introduction to generative ai," Seminars in Computer Engineering, Universidade Estadual de Campinas, 2025. [Online]. Available: https://feec-seminar-comp-eng.github.io/seminars/seminars-1-2024/6/

[2] Google, "Notebooklm: Ai-powered research and learning assistant," Online Tool, 2025. [Online]. Available: https://notebooklm.google/

[3] Google, "Gemini cli," GitHub Repository, 2025. [Online]. Available: https://github.com/google-gemini/gemini-cli